

Базы данных

Лекция 08 – Berkeley DB. Управление файлами БД

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2025

Оглавление

Управление файлами базы данных.....	3
Контрольные точки для окружений (Checkpoints).....	4
DB_ENV->txn_checkpoint() – запустить контрольную точку.....	6
Процедуры резервного копирования.....	11
Об утилитах копирования Unix.....	13
Резервные копии в автономном режиме (Offline Backups).....	14
Горячее резервное копирование (Hot Backup).....	15
Инкрементальное резервное копирование (Incremental Backups).....	16
Процедуры восстановления (Recovery Procedures).....	17
Нормальное восстановление (Normal Recovery).....	18
Катастрофическое восстановление (Catastrophic Recovery).....	21
Проектирование приложения для восстановления.....	24
Восстановление для многопоточных приложений.....	24
Восстановление в многопроцессных приложениях.....	27
DB_ENV->set_thread_count().....	33
DB_ENV->set_isalive().....	35
DB_ENV->set_thread_id().....	37
DB_ENV->failchk().....	39
Использование горячих аварийных переключений (Using Hot Failovers).....	42
Удаление файлов журнала (Removing Log Files).....	45
Настройка подсистемы ведения журнала.....	47
Установка размера файла журнала (Setting the Log File Size).....	48
Настройка размера области ведения журнала (Configuring the Logging Region Size).....	50
Настройка ведения журнала в памяти (ACI).....	51
Настройка размера буфера журнала в памяти (Setting In-Memory Log Buffer Size).....	55
Резюме и примеры.....	56
Анатомия транзакционного приложения.....	56
Пример транзакции.....	59
Пример транзакции в памяти.....	81

Управление файлами базы данных

BDB может хранить на диске несколько типов файлов:

- 1) файлы данных, которые содержат фактические данные в базе данных;
- 2) файлы журналов, которые содержат информацию, необходимую для восстановления базы данных в случае сбоя системы или приложения.
- 3) файлы областей¹, которые содержат информацию, необходимую для общей работы приложения.
- 4) временные файлы, которые создаются только при определенных особых обстоятельствах. Эти файлы никогда не нужно ни резервировать ни управлять каким либо иным образом.

Из вышеупомянутых требуют управления пользователем только файлы данных и файлы журналов, обеспечивая их резервное копирование.

Также следует обращать внимание на объем дискового пространства, которое занимают файлы журналов, и периодически удалять ненужные файлы.

Также можно по желанию настроить подсистему журналирования, чтобы она наилучшим образом соответствовала потребностям и требованиям приложения.

1) region files

Контрольные точки для окружений (Checkpoints)

Когда изменяются базы данных (фиксируется транзакция), изменения записываются в журналы БД, но они не обязательно отражаются в реальных файлах базы данных на диске.

Это означает, что с течением времени в файлах журналов содержится все больше данных, которые пока еще не попали в файлы с данными. В результате приходится хранить больше файлов журналов, чем может быть необходимо.

Кроме того, любое восстановление из файлов журналов будет занимать все больше времени, поскольку в файлах журналов содержится большое количество данных, которые должны быть восстановлены и попасть в файлы данных.

Можно уменьшить эти проблемы, периодически запуская «контрольные точки» для окружения.

Контрольная точка:

1) Очищает грязные страницы из кэша в памяти. Это означает, что изменения данных, обнаруженные в кэше в памяти, записываются в файлы базы данных на диске.

Контрольная точка также приводит к тому, что данные, загрязненные незафиксированной транзакцией, также записываются в файлы базы данных на диск. В этом последнем случае для удаления любых таких изменений, которые впоследствии были отменены приложением с помощью прерывания транзакции, используется обычное восстановление DB.

(Процедуры восстановления, стр. 17).

2) Выполняет запись контрольной точки.

3) Очищает журнал. Это приводит к записи всех данных в журнале, которые еще не были записаны на диск.

4) Записывает список открытых баз данных.

Есть несколько способов запустить контрольную точку. Один из способов — использовать утилиту командной строки **db_checkpoint(1)**.

Однако следует отметить, что эту утилиту командной строки нельзя использовать, если окружение была открыта с помощью **DB_PRIVATE**.

Также можно запустить поток, который периодически проверяет окружение, вызывая метод **DB_ENV->txn_checkpoint()**.

При необходимости можно предотвратить выполнение контрольной точки, если с момента последней контрольной точки не было записано больше указанного количества данных журнала.

Также можно предотвратить выполнение контрольной точки, если с момента последней контрольной точки не прошло больше указанного количества времени.

Эти условия особенно интересны, если есть несколько потоков или процессов, выполняющих запуск контрольных точек.

Запуск контрольных точек может быть довольно затратным.

1) БД должна сбрасывать каждую грязную страницу в файлы резервной базы данных.

2) С другой стороны, если контрольные точки не запускаются достаточно часто, время восстановления может быть неоправданно долгим, и может использовать больше дискового пространства, чем действительно нужно.

3) Кроме того, пока не будет запущена контрольная точка, невозможно удалить файлы журналов.

Поэтому решение о том, как часто запускать контрольную точку, является одним из наиболее распространенных действий по настройке приложений БД.

DB_ENV->txn_checkpoint() – запустить контрольную точку

```
#include <db.h>
int DB_ENV->txn_checkpoint(const DB_ENV *env,
                           u_int32_t      kbyte,  // лимит данных
                           u_int32_t      min,    // лимит времени
                           u_int32_t      flags); // флаг DB_FORCE или 0
```

Если с момента последней контрольной точки в среде базы данных велась какая-либо активность журналирования, метод DB_ENV->txn_checkpoint() очищает базовый пул памяти, записывает запись о контрольной точке в журнал, а затем очищает журнал.

Метод DB_ENV->txn_checkpoint() возвращает ненулевое значение ошибки в случае ошибки и 0 в случае успешного выполнения.

Метод DB_ENV->txn_checkpoint() является базовым методом, используемым утилитой **db_checkpoint**.

Пример использования DB_ENV->txn_checkpoint() в среде IEEE/ANSI Std 1003.1 (POSIX) можно найти в исходном коде утилиты db_checkpoint.

Параметры

kbyte — если параметр kbyte не равен нулю, контрольная точка будет создана, если с момента последней контрольной точки было записано более kbyte килобайт данных журнала.

min — если параметр min не равен нулю, контрольная точка будет создана, если с момента последней контрольной точки прошло более min минут.

flags — параметр `flags` должен быть равен 0 или следующему значению:

DB_FORCE – принудительно записать контрольную точку, даже если с момента последней контрольной точки не было активности.

Ошибки

Метод `DB_ENV->txn_checkpoint( )` может завершиться ошибкой и вернуть одну из следующих ненулевых ошибок:

EINVAL — указано недопустимое значение флага или параметра.

Пример запуска контрольной точки из отдельного потока управления:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <string.h>
#include "db.h"

void* checkpoint_thread(void *); // поток с запуском контрольной точки

int main(void) {

    int          ret;
    u_int32_t    env_flags;
    DB_ENV      *envp = NULL;
    const char *db_home_dir = "/tmp/myEnvironment";
    pthread_t    ptid;

    // Создаем окружение
    ret = db_env_create(&envp, 0);
    if (ret != 0) {
        fprintf(stderr,
                "Ошибка создания дескриптора окружения: %s\n",
                db_strerror(ret));
        return (EXIT_FAILURE);
    }
}
```



```

env_flags =
    DB_CREATE |      // Создать окружение, если не существует
    DB_INIT_LOCK |   // Инициализация блокировок
    DB_INIT_LOG |    // Инициализация журналирования
    DB_INIT_MPOOL |  // Инициализация кеширования в памяти
    DB_THREAD |      // Free-thread the env handle
    DB_INIT_TXN;     // Инициализация транзакций

// Открываем окружение
ret = envp->open(envp, db_home_dir, env_flags, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка открытия окружения: %s\n",
               db_strerror(ret));
    goto err;
}
// Старт потока с контрольной точкой
ret = pthread_create(&ptid, NULL, checkpoint_thread, (void *)envp);
if (ret != 0) {
    fprintf(stderr,
            "txnapp: failed spawning checkpoint thread: %s\n",
            db_strerror(ret));
    goto err;
}
// All other threads and application shutdown code omitted for brevity.
err:
    ...
}

```

```
void* checkpoint_thread(void *arg) {  
    DB_ENV *dbenv;  
    int ret;  
    dbenv = arg;  
    // Checkpoint раз в минуту  
    for (;;) sleep(60)) {  
        if ((ret = dbenv->txn_checkpoint(dbenv, 0, 0, 0)) != 0) {  
            dbenv->err(dbenv, ret, "checkpoint thread");  
            exit(1);  
        }  
    }  
    /* NOTREACHED */  
}
```

Процедуры резервного копирования

Важной частью транзакционных гарантий является долговечность (Durability).

Это означает, что после успешного завершения транзакции приложение всегда будет видеть результаты этой транзакции.

Ни один программный алгоритм не может гарантировать долговечность в условиях физической потери данных.

Жесткие диски могут выйти из строя, и если данные не скопированы в места, отличные от основных дисковых накопителей, то при выходе этих накопителей из строя данные определенно будут потеряны.

Поэтому, чтобы действительно получить гарантию долговечности, необходимо убедиться, что все данные, хранящиеся на диске, резервируются во вторичном или альтернативном хранилище, например, на вторичных дисковых накопителях или автономных лентах.

Существует три различных типа резервного копирования, которые можно выполнять с базами данных и файлами журналов:

- резервное копирование в автономном режиме (Offline backups);
- горячее создание резервных копий (Hot backups);
- инкрементное создание резервных копий (Incremental backups).

1) Резервное копирование в автономном режиме (Offline backups)

Этот тип резервного копирования, пожалуй, самый простой в выполнении, поскольку он включает в себя простое копирование базы данных и файлов журнала в область автономного хранения. Он также дает снимок базы данных в фиксированный, известный момент времени. Однако во время выполнения записи в базу данных этот тип резервного копирования выполнить невозможно.

2) Горячее создание резервных копий (Hot backups)

Этот тип резервного копирования также дает снимок базы данных. Поскольку приложение может писать в базу данных во время создания снимка, точное состояние базы данных для этого снимка неизвестно.

3) Инкрементное создание резервных копий (Incremental backups)

Этот тип резервного копирования обновляет ранее выполненную резервную копию.

Примечание

Резервные копии баз данных не зависят от порядка байтов. То есть резервная копия, сделанная на машине с обратным порядком байтов, может быть использована для восстановления базы данных, находящейся на машине с прямым порядком байтов.

После выполнения резервного копирования можно выполнить и катастрофическое восстановление, чтобы восстановить базы данных из резервной копии. (Катастрофическое восстановление, стр.21).

Следует обратить внимание, что также можно поддерживать горячее аварийное переключение.

(Использование горячих аварийных переключений, стр.Ошибка: источник перекрёстной ссылки не найден).

Об утилитах копирования Unix

Если копируются файлы базы данных, они должны копироваться **атомарно, кратно размеру страницы базы данных**.

Это значит, что чтения, выполняемые программой копирования, не должны чередоваться с записями других потоков управления, и программа копирования должна считывать базы данных кратно размеру страницы базовой базы данных.

Как правило, операционные системы уже гарантируют это, а системные утилиты обычно считывают фрагментами размером степени двойки, которые больше максимально возможного размера страницы базы данных Berkeley DB.

На некоторых платформах (в частности, в некоторых выпусках Solaris) утилита копирования (`cp`) была реализована с использованием системного вызова `mmap()`, а не системного вызова `read()`. Поскольку `mmap()` не давал той же гарантии атомарности чтения, что `read()`, утилита `cp` могла создавать поврежденные копии баз данных.

Кроме того, на некоторых платформах есть реализации утилиты `tar`, которая по умолчанию выполняет чтение блоков по 10 КБ. Даже если указан размер выходного блока, утилита все равно не будет читать базу данных блоками, кратными размеру блока БД. Соответственно, результатом может стать поврежденная резервная копия.

Чтобы избежать возможных проблемы, следует использовать утилиту **`dd(1)`** вместо **`cp(1)`** или **`tar(1)`**. При использовании **`dd(1)`** следует обязательно указать размер блока, равный или даже кратный размеру страницы БД. При использовании системных утилит для копирования файлов базы данных, имеет смысл использовать утилиту трассировки системных вызовов (`ktrace`, `truss`, `strace`), чтобы убедиться, что не используется размер ввода-вывода, меньший размера страницы БД. Также можно использовать эти утилиты, чтобы убедиться, что системная утилита не использует системный вызов, отличный от `read()`.

Резервные копии в автономном режиме (Offline Backups)

Чтобы создать резервную копию в автономном режиме следует:

- 1) зафиксировать или отменить все текущие транзакции.
- 2) приостановить все записи в базу данных.
- 3) принудительно создать контрольную точку. (Контрольные точки для окружений (Checkpoints)).
- 4) скопировать все файлы базы данных в место хранения резервной копии.

Можно скопировать все файлы базы данных или определить, какие файлы базы данных были записаны в течение жизненного цикла текущих журналов.

Для этого используется метод **DB_ENV->log_archive()** с опцией **DB_ARCH_DATA**, либо команда **db_archive** с опцией **-s**.

Резервное копирование только что измененных баз данных работает только в том случае, если есть все файлы журналов.

Если файлы журналов по какой-либо причине были удалены, то использование `log_archive()` может привести к невозстановимой резервной копии.

- 5) Скопировать *последний* файл журнала в резервную копию.

Файлы журнала имеют имена `log.xxxxxxxxxx`, где `xxxxxxxxxx` — это порядковый номер. Последний файл журнала — это файл с самым большим номером.

Горячее резервное копирование (Hot Backup)

Чтобы реализовать горячее резервное копирование, не нужно останавливать операции в базе данных. Транзакции могут быть текущими, и во время резервного копирования можно записывать данные в базу. Однако это означает, что никто не знает точно, в каком состоянии находится база данных во время резервного копирования.

Можно использовать утилиту командной строки **db_hotbackup** для создания горячего резервного копирования. Эта программа опционально запускает контрольную точку, а затем копирует все необходимые файлы в целевой каталог.

Также можно создать свой собственный объект горячего резервного копирования с помощью метода **DB_ENV->backup()**.

Кроме того, можно вручную создать горячее резервное копирование:

1. Следует установить флаг **DB_HOTBACKUP_IN_PROGRESS** в окружении, используя метод **DB_ENV->set_flags()**.

2. Следует скопировать все файлы базы данных в резервную копию.

Можно скопировать все файлы базы данных или определить, какие файлы базы данных были записаны в течение жизненного цикла текущих журналов. Для этого следует использовать либо **DB_ENV->log_archive()** с опцией **DB_ARCH_DATA**, либо команду **db_archive(1)** с опцией **-s**.

3. Следует скопировать все журналы в резервное хранилище.

4. Следует сбросить флаг **DB_HOTBACKUP_IN_PROGRESS**.

Важно скопировать файлы базы данных, и только потом затем журналы.

Таким образом можно завершить или откатить любые операции с базой данных, которые были выполнены лишь частично при копировании баз данных.

Инкрементальное резервное копирование (Incremental Backups)

После того, как создана полная резервная копия (то есть обычная или горячая), можно создавать инкрементальные резервные копии. Для этого просто следует скопировать все текущие файлы журналов в место хранения резервной копии.

Инкрементальные резервные копии не требуют запуска контрольной точки или прекращения операций записи в базу данных.

При работе с инкрементальными резервными копиями следует помнить, что чем больше файлов журналов содержится в резервной копии, тем больше времени займет восстановление.

Следует запускать полное резервное копирование с некоторым интервалом, а затем выполнять инкрементальное резервное копирование с более коротким интервалом.

Частота, с которой нужно запускать полное резервное копирование, определяется скоростью изменения баз данных и чувствительностью приложения к длительному восстановлению (если оно требуется).

Также можно сократить время восстановления, запуская восстановление из резервной копии по мере создания каждой инкрементной резервной копии. Запуск восстановления по мере выполнения означает, что у DB будет меньше работы, если когда-либо понадобится восстановить окружение из резервной копии.

Процедуры восстановления (Recovery Procedures)

БД поддерживает два типа восстановления:

1) Обычное восстановление, которое запускается, когда окружение открывается при запуске приложения, проверяет только те записи журнала, которые необходимы для приведения баз данных в согласованное состояние с момента ***последней контрольной точки***.

Обычное восстановление начинается с журналов, которые использовались транзакциями, активными во время последней контрольной точки, и проверяются все журналы с этого момента до текущих журналов.

2) Катастрофическое восстановление выполняется так же, как и обычное восстановление, за исключением того, что оно проверяет все доступные файлы журналов. Катастрофическое восстановление используется для восстановления баз данных из ***ранее созданной резервной копии***.

Обычное восстановление следует запускать всякий раз, когда запускается приложение.

Катастрофическое восстановление запускается всякий раз, когда потеряны или повреждены файлы базы данных и необходимо восстановить их из резервной копии.

Нормальное восстановление (Normal Recovery)

1) Нормальное восстановление проверяет содержимое файлов журнала окружения и использует эту информацию, чтобы убедиться, что файлы базы данных согласованы с информацией, содержащейся в файлах журнала.

2) Нормальное восстановление также воссоздает файлы из области окружения.

3) Оно также очищает любые неразблокированные блокировки, которые приложение могло удерживать во время некорректного завершения работы приложения.

Нормальное восстановление запускается только для тех файлов журнала, которые были созданы с момента *последней контрольной точки*. По этой причине время восстановления зависит от того, сколько данных было записано с момента последней контрольной точки, и, следовательно, от того, сколько информации в файле журнала нужно изучить.

Если контрольные точки запускаются нечасто, то нормальное восстановление может занять относительно много времени.

Примечание

Следует запускать нормальное восстановление каждый раз, когда выполняется запуск приложения.

Чтобы запустить нормальное восстановление, следует:

1) убедиться, что все дескрипторы окружения закрыты;

2) нормальное восстановление должно быть однопоточным;

3) указать флаг **DB_RECOVER** при открытии окружения.

Также можно запустить восстановление, приостановив или завершив работу приложения и используя утилиту командной строки **db_recover(1)**.

SIGSTOP/SIGCONT — остановить/продолжить, если остановлен (`kill -s SIG* pid`).

Пример

```
#include <stdio.h>
#include <stdlib.h>
#include "db.h"

int main(void) {

    int ret;
    u_int32_t env_flags;
    DB_ENV *envp = NULL;
    const char *db_home_dir = "/tmp/myEnvironment";
    // создаем дескриптор
    ret = db_env_create(&envp, 0);
    if (ret != 0) {
        fprintf(stderr, "Ошибка создания дескриптора окружения: %s\n",
                db_strerror(ret));
        return (EXIT_FAILURE);
    }
    // указываем флаг восстановления для окружения
    env_flags = DB_CREATE | // Создать окружение, если не существует
                DB_INIT_LOCK | // Инициализация блокировок
                DB_INIT_LOG | // Инициализация журналирования
                DB_INIT_MPOOL | // Инициализация кеширования в памяти
                DB_THREAD | // Free-thread the env handle
                DB_INIT_TXN | // Инициализация транзакций
                DB_RECOVER; // Run normal recovery
```

```
// и открываем его
ret = envp->open(envp, db_home_dir, env_flags, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка открытия окружения: %s\n",
               db_strerror(ret));
    goto err;
}
// Все остальные операции отсюда идентичны.
// Следует обратить внимание, что нельзя создавать никаких других потоков
// управления до завершения восстановления.
```

Катастрофическое восстановление (Catastrophic Recovery)

Катастрофическое восстановление используется, когда восстанавливаются базы данных из ранее созданной резервной копии.

Для восстановления баз данных из предыдущей резервной копии следует скопировать всю резервную копию **в новый каталог окружения**, а затем запустить катастрофическое восстановление.

Невыполнение этого требования может привести к тому, что внутренние структуры базы данных не будут синхронизированы с файлами журналов.

Катастрофическое восстановление должно быть запущено в однопоточном режиме.

Чтобы запустить катастрофическое восстановление, следует:

- 1) завершить все операции с базой данных;
- 2) восстановить резервную копию в пустой каталог;
- 3) указать флаг **DB_RECOVER_FATAL** при открытии окружения.

Эта открытое окружение должно быть однопоточным.

Также можно запустить восстановление, приостановив или завершив работу приложения и использовать утилиту командной строки **db_recover** с опцией **-c** (catastrophic).

Катастрофическое восстановление проверяет каждый доступный файл журнала, а не только те файлы журнала, которые были созданы с момента последней контрольной точки, как в случае обычного восстановления. По этой причине катастрофическое восстановление, скорее всего, займет больше времени, чем обычное восстановление.

```
#include <stdio.h>
#include <stdlib.h>
#include "db.h"

int main(void) {

    int ret;
    u_int32_t env_flags;
    DB_ENV *envp = NULL;
    const char *db_home_dir = "/tmp/myEnvironment";

    // создаем дескриптор для окружения
    ret = db_env_create(&envp, 0);
    if (ret != 0) {
        fprintf(stderr, "Ошибка создания дескриптора окружения: %s\n",
                    db_strerror(ret));
        return (EXIT_FAILURE);
    }
    // Указываем во флагах, что необходимо запустить восстановление
    env_flags =
        DB_CREATE |           // Создать окружение, если не существует
        DB_INIT_LOCK |       // Инициализация блокировок
        DB_INIT_LOG |        // Инициализация журналирования
        DB_INIT_MPOOL |      // Инициализация кеширования в памяти
        DB_THREAD |          // Free-thread the env handle
        DB_INIT_TXN |         // Инициализация транзакций
        DB_RECOVER_FATAL;    // Run catastrophic recovery
```

```
// Открываем окружение
ret = envp->open(envp, db_home_dir, env_flags, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка открытия окружения: %s\n",
               db_strerror(ret));
    goto err;
}
// Все остальные операции отсюда обычные.
```